

WORKING WITH DATE, OBJECT, ARRAY

- ❖ Date
- ❖ Object
- ❖ Array



DATE

- ❖ Date: là một kiểu dữ liệu sử dụng để lưu trữ thông tin về mốc thời gian như: ngày, tháng, năm ...
- ❖ Mốc thời gian được lưu sẽ tính theo múi giờ của trình duyệt, dựa trên vị trí của người dùng, ở VN sẽ là GMT+7
- ❖ Có thể sử dụng: `Date.now()` để có được tổng số mili giây tính từ năm 01/01/1970

```
const milliseconds = Date.now();
```

- ❖ Có thể tạo một biến kiểu date để lưu trữ một mốc thời gian mong muốn bằng cú pháp: `new Date()`
 - `new Date()`: tạo đối tượng kiểu date lưu trữ thời gian hiện tại
 - `new Date(year, month, day, hours, minutes, seconds, milliseconds)`: tạo ra một đối tượng kiểu date dựa trên những thông tin được truyền vào, lưu ý tháng, giờ, phút, giây, mili giây trong JS khi ở dạng số sẽ bắt đầu từ 0 thay vì 1, có vài cách truyền các thông tin này tùy theo nhu cầu, đây là dạng đầy đủ nhất
 - `new Date(milliseconds)`: tạo ra một đối tượng kiểu date dựa trên số mili giây từ 01/01/1970 đến thời điểm đó
 - `new Date(date string)`: tạo ra một đối tượng kiểu date dựa trên một chuỗi diễn tả được khoảng thời gian mong muốn, cần đúng một số format nhất định, có nhiều format được JS chấp nhận
- => Tìm hiểu thêm

```
const myDate = new Date(value: "2017/03/04");
```

DATE

- ❖ Khi đã có được một đối tượng lưu dữ liệu date, ta có thể sử dụng một số thuộc tính của chúng để làm việc
 - `getFullYear()`: lấy ra năm của mốc thời gian (yyyy)
 - `getMonth()`: lấy ra tháng của mốc thời gian (0-11)
 - `getDate()`: lấy ra ngày trong tháng của mốc thời gian (1-31)
 - `getHours()`: lấy ra giờ của mốc thời gian (0-23)
 - `getMinutes()`:
 -
 - `getTime()`: lấy ra số mili giây tính từ 01/01/1970: giống `Date.now()`
 - `getDay()`: lấy ra ngày trong tuần (0-6) : 0 là chủ nhật
- ❖ Ta cũng có thể thay đổi dữ liệu đang được lưu trong đối tượng đó bằng các hàm
 - `setDate()`
 - `setFullYear()`
 - `setHours()`
 -

OBJECT

- ❖ Thao tác với Object ta có thể:
 - Sử dụng vòng lặp for-in để tạo vòng lặp qua các key của object
 - `Object.values(a)`: trả về một mảng là danh sách các giá trị của các key có trong object a
 - `Object.keys(a)`: trả về một mảng là danh sách các key có trong a
 - `Object.entries(a)`: trả về một mảng là danh sách các cặp key-value có trong a

ARRAY

- ❖ Một số hàm có thể được sử dụng để thao tác, chỉnh sửa các phần tử của array
- ❖ Các hàm này làm thay đổi dữ liệu mà array đang lưu trữ tùy vào nhu cầu
 - `arr.join(a)`: tạo ra một chuỗi, chuỗi này được tạo ra bằng cách cố gắng đưa các phần tử trong mảng `arr` thành kiểu dữ liệu string, ghép chúng lại và cách nhau bởi chuỗi `a`, mặc định là dấu “ ”
 - `Arr.pop()`: thay đổi mảng bằng cách lấy ra phần tử cuối cùng của mảng `arr` và trả về phần tử đó
 - `Arr.shift()`: như `pop()` nhưng lấy ra phần tử đầu tiên
 - `Arr.push(a)`: thay đổi mảng bằng cách thêm phần tử `a` vào cuối mảng
 - `Arr.unshift(a)`: thay đổi mảng bằng cách thêm phần tử `a` vào đầu mảng
 - `arr1.concat(arr2, arr3, ...)`, tạo ra một mảng mới bằng cách nối các mảng `arr1`, `arr2`, `arr3` lại với nhau
 - Một số hàm khác:
 - `splice()`
 - `slice()`

ARRAY ADVANCE

❖ Giới thiệu về các hàm xử lý array

- `Array.forEach()`
- `Array.map()`
- `Array.filter()`
- `Array.find()`
- `Array.every()`
- `Array.some()`
- `Array.sort()`
- `Array.reduce()`

forEach

- ❖ Hàm forEach được sử dụng khi ta muốn làm một việc gì đó với từng phần tử của mảng (giống như một vòng lặp)
- ❖ Hàm forEach là hàm đơn giản nhất, hàm callback mà nó cần chỉ đơn giản là làm một việc bất kỳ, nó chỉ thực hiện hàm callback này trên từng phần tử của mảng mà không quan tâm đến kết quả của hàm đó
- ❖ Trong trường hợp hàm của ta không dùng đến index hoặc array, ta có thể bỏ qua chúng khi định nghĩa hàm callback
- ❖ Hàm forEach không trả về giá trị

Map

- ❖ Hàm map của array được sử dụng để tạo ra một array mới với các phần tử được **tạo** ra từ các phần tử có vị trí tương ứng của array cũ => array mới sẽ có số phần tử luôn bằng số phần tử của array cũ
- ❖ Lần lượt các phần tử của array cũ được đưa qua hàm callback, kết quả trả về của hàm callback này sẽ là dữ liệu của phần tử có vị trí tương ứng trong array mới
- ❖ Hàm callback ở đây phải mô tả được cách ta tạo ra phần tử mới từ các thông tin được cung cấp, phải đảm bảo luôn trả về một giá trị, nếu không, mặc định phần tử có vị trí tương ứng trong mảng mới sẽ là undefined, vì mặc định một hàm luôn trả về undefined
- ❖ Hàm map trả về một mảng mới mà không làm ảnh hưởng đến mảng cũ
- ❖ Ví dụ:
 - Cho một mảng [1,2,3,4,5], tạo ra một mảng có các phần tử là: [2,4,6,8,10]
 - Tạo một mảng học sinh mới, với mỗi học sinh có thêm một thông tin là học lực dựa trên dữ liệu điểm của học sinh đó

ARRAY FILTER

- ❖ Hàm filter của array được sử dụng để tạo ra một array mới với các phần tử được **lọc** ra từ các phần tử có vị trí tương ứng của array cũ => array mới sẽ có số phần tử luôn bằng hoặc nhỏ hơn số phần tử của array cũ
- ❖ Lần lượt các phần tử của array cũ được đưa qua hàm callback, kết quả trả về của hàm callback này là truthy thì có nghĩa phần tử này được phép đưa vào array mới, ngược lại nếu kết quả của hàm callback là falsy thì phần tử sẽ bị bỏ qua
- ❖ Hàm callback ở đây phải mô tả được cách ta kiểm tra xem một phần tử có được phép đưa vào mảng mới không, hàm callback luôn phải trả về kết quả nếu không mặc định sẽ trả về undefined => falsy
- ❖ Hàm filter trả về một mảng mới mà không làm ảnh hưởng đến mảng cũ
- ❖ Ví dụ:
 - Cho một mảng [1,2,3,4,5,6] , tạo ra một mảng có các phần tử là: [2,4,6]
 - Tạo một mảng học sinh mới, chỉ chứa các học sinh có điểm tổng kết ≥ 8

ARRAY FIND

- ❖ Hàm find của array được sử dụng để **tìm kiếm** ra một phần tử đầu tiên thỏa mãn một điều kiện nào đó=> kết quả trả về là một phần tử của array hoặc undefined khi không tìm thấy phần tử nào thỏa mãn
- ❖ Lần lượt các phần tử của array được đưa qua hàm callback, kết quả trả về của hàm callback này là truthy thì có nghĩa phần tử này thỏa mãn điều kiện và việc tìm kiếm dừng lại, ngược lại nếu kết quả của hàm callback là falsy thì phần tử sẽ bị bỏ qua để thử với phần tử tiếp theo
- ❖ Hàm callback ở đây phải mô tả được cách ta kiểm tra xem một phần tử có thỏa mãn không, hàm callback luôn phải trả về kết quả nếu không mặc định sẽ trả về undefined => falsy
- ❖ Hàm find trả về một phần tử của mảng ban đầu hoặc undefined mà không làm ảnh hưởng đến mảng cũ
- ❖ Ví dụ:
 - Cho một mảng [6,9,10,11,15,16] , trả về số nguyên tố đầu tiên
 - Tìm học sinh đầu tiên có điểm tổng kết ≥ 8

ARRAY SOME VÀ EVERY

- ❖ Hàm some và every của array được sử dụng để **kiểm tra** xem các phần tử của mảng có thỏa mãn điều kiện nào đó hay không => kết quả trả về là true hoặc false
 - Hàm some: trả về true khi có **ít nhất một phần** tử thỏa mãn điều kiện và ngược lại
 - Hàm every: trả về true khi **tất cả các phần tử** thỏa mãn điều kiện và ngược lại
- ❖ Lần lượt các phần tử của array được đưa qua hàm callback, kết quả trả về của hàm callback này là truthy thì có nghĩa phần tử này thỏa mãn điều kiện, ngược lại nếu kết quả của hàm callback là falsy thì được coi là không thỏa mãn điều kiện
- ❖ Hàm callback ở đây phải mô tả được cách ta kiểm tra xem một phần tử có thỏa mãn không, hàm callback luôn phải trả về kết quả nếu không mặc định sẽ trả về undefined => falsy
- ❖ Hàm some và every trả về true hoặc false mà không làm ảnh hưởng đến mảng cũ
- ❖ Ví dụ:
 - Cho một mảng [6,9,10,11,15,16] , kiểm tra xem có phải tất cả là số chẵn không , kiểm tra có ít nhất một số nguyên tố hay không
 - Kiểm tra có phải tất cả học sinh đều qua môn không (điểm tổng kết trên 4)

ARRAY SORT

- ❖ Hàm sort của array được sử dụng để **sắp xếp** các phần tử của mảng theo một thứ tự nào đó, mặc định khi sắp xếp một mảng là chữ, hàm sort không cần nhận hàm callback, có thể sort “apple” đứng trước “banana” vì ở đây nó coi các phần tử là một chuỗi nên sẽ tự sắp xếp theo alphabet
- ❖ Khi muốn thực hiện sắp xếp số hoặc một kiểu dữ liệu khác, ta cần cho hàm sort biết khi nào thì phần tử a đứng trước phần tử b và ngược lại,
- ❖ Lần lượt các cặp phần tử (a,b) của array được đưa qua hàm callback, kết quả trả về của hàm callback này sẽ xác định xem phần tử a có đứng trước b không, nếu trong mảng ban đầu thứ tự này bị sai, hàm sort sẽ đảo chỗ lại cho đúng
 - Kết quả trả về của hàm callback > 0 : a phải đứng trước b
 - Kết quả trả về của hàm callback < 0 : b phải đứng trước a
 - Kết quả trả về của hàm callback $= 0$: không cần thay đổi vị trí của a và b
- ❖ Hàm callback ở đây phải mô tả được cách ta kiểm tra xem một phần tử có đang đứng đúng vị trí tương đối so với một phần tử khác hay không
- ❖ Hàm sort làm thay đổi mảng ban đầu và trả về chính mảng đó
- ❖ Ví dụ:
 - Cho một mảng [6,9,10,11,15,16], sắp xếp theo thứ tự tăng dần, giảm dần
 - Sắp xếp danh sách học sinh theo điểm từ cao đến thấp
 - Sắp xếp danh sách học sinh theo điểm tổng ba môn toán, văn, anh, nếu 2 học sinh có điểm 3 môn bằng nhau, sắp xếp theo điểm toán

Hàm	Mục Đích	Trả Về
<code>`map`</code>	Tạo một mảng mới với kết quả của việc gọi một hàm cung cấp trên mỗi phần tử của mảng.	Một mảng mới với các phần tử đã được biến đổi.
<code>`forEach`</code>	Thực thi một hàm cung cấp một lần cho mỗi phần tử trong mảng.	<code>`undefined`</code> (chỉ có tác dụng phụ).
<code>`every`</code>	Kiểm tra xem tất cả các phần tử trong mảng có đáp ứng điều kiện được cung cấp hay không.	<code>`true`</code> nếu tất cả các phần tử đáp ứng điều kiện, <code>`false`</code> nếu ngược lại.
<code>`some`</code>	Kiểm tra xem ít nhất một phần tử trong mảng có đáp ứng điều kiện được cung cấp hay không.	<code>`true`</code> nếu ít nhất một phần tử đáp ứng điều kiện, <code>`false`</code> nếu ngược lại.
<code>`filter`</code>	Tạo một mảng mới với tất cả các phần tử đáp ứng điều kiện được cung cấp.	Một mảng mới với các phần tử đáp ứng điều kiện.
<code>`find`</code>	Trả về phần tử đầu tiên trong mảng thỏa mãn điều kiện kiểm tra được cung cấp.	Phần tử đầu tiên thỏa mãn, hoặc <code>`undefined`</code> nếu không có phần tử nào thỏa mãn.
<code>`reduce`</code>	Áp dụng một hàm lên một bộ tích lũy và mỗi phần tử trong mảng để giảm nó thành một giá trị duy nhất.	Một giá trị duy nhất đã được tích lũy.
<code>`sort`</code>	Sắp xếp các phần tử của mảng tại chỗ và trả về mảng đã sắp xếp.	Mảng đã được sắp xếp.